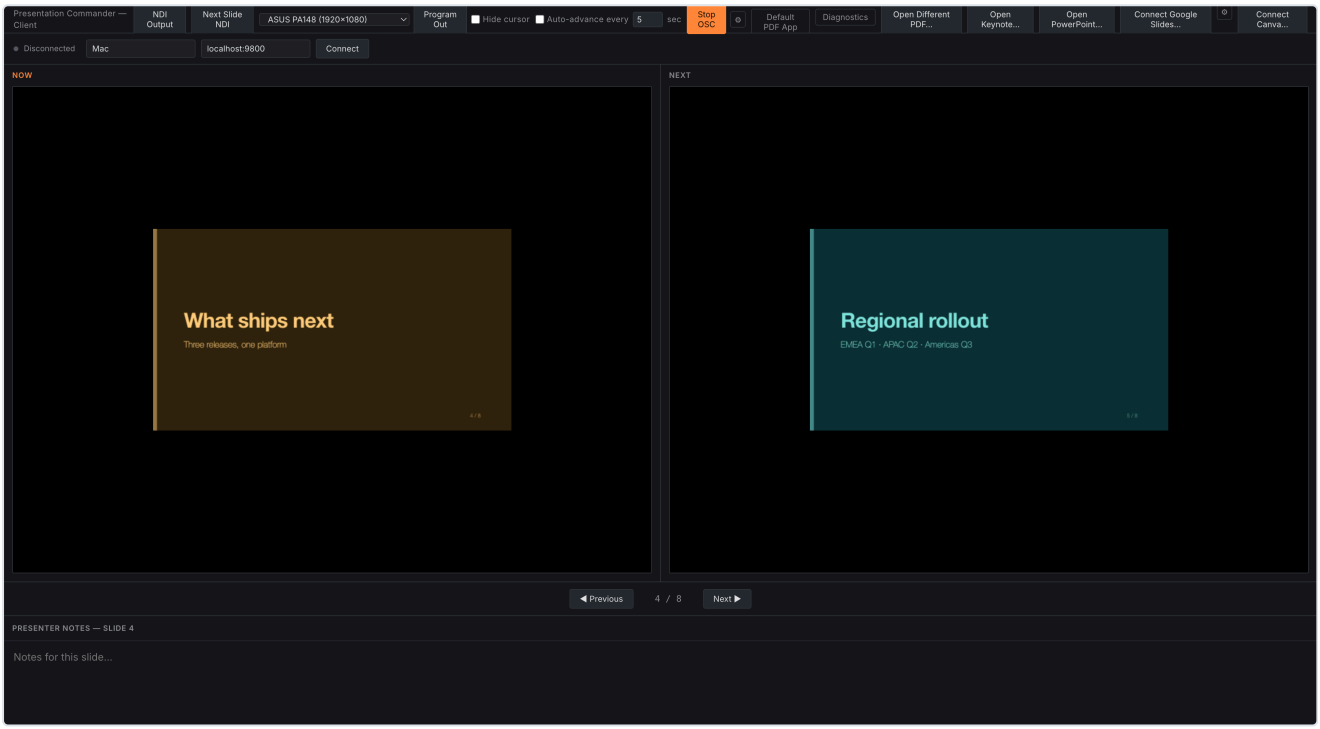
Presentation Commander Client user guide

Presentation Commander Client runs on the presentation laptop. It puts the deck on screen, sends it out over NDI, keeps presenter notes, and takes remote control from a Stream Deck or the Master Server.



The main window presenting a real PDF: Now and Next, with the Program Out display picker above.

The README covers what each feature is and how to install it. This is how to run a show with it, and what'll catch you out.

Choose your source first — the capabilities differ

Everything else follows from this choice, because what a source can do depends on what the application underneath actually lets automation get at. I've researched each of these limits, and none of them is a feature I just haven't written yet.

	PDF	Keynote (mac)	PowerPoint (mac)	PowerPoint (Win)	Google Slides	Canva
Works on Windows	✅	—	—	✅	✅	✅
Works on Linux	✅	—	—	—	✅	✅
Drives the real app	—	✅	✅	✅	✅	✅
Follows the presenter's own clicker	—	✅	✅	✅	✅	✅
Clickable internal links (TOC, "back to agenda")	✅	—	—	—	—	—
Sections for `goto/section`	✅	—	—	✅	—	—
Media play/pause/stop	—	—	—	✅[1]	—	—
Live screen capture (animations, video)	—	✅	✅	✅	—	—
Needs the browser extension	—	—	—	—	✅	✅
Needs OAuth setup	—	—	—	—	✅	—

[1] Only while a **live PowerPoint slideshow is running independently of this app**, and all four media commands are the same keyboard toggle underneath. There's no separate play and pause.

On Linux you won't see "Open Keynote…" or "Open PowerPoint…". Driving either one needs AppleScript or Windows COM automation, and Linux has nothing equivalent, so the app hides the two buttons rather than showing controls that can't work. PDF, Google Slides and Canva are all available there.

The PDF engine is the most capable and the most predictable. It's the only source with clickable internal links, it works the same on all three platforms, and it never depends on another application staying responsive. If you can export the deck to PDF, do, and use presentation-converter if you want the presenter notes to come across with it (Presenter notes).

Nothing drives PowerPoint's or Keynote's own fullscreen slideshow mode. It was confirmed unreliable under automation and virtualization on both platforms. This app drives the *editing view* and puts the audience picture out through Program Out / NDI instead. That's what the audience actually sees, so you don't lose anything by it.

Program Out

A second, fullscreen, chrome-free window showing only the current slide, for a projector or confidence monitor. Pick which connected display it opens on from the dropdown.

- `B` / `W` blank it to solid black or white without losing your place, using the same shortcuts as PowerPoint. Black and white are one state: turning on white clears black.
- An optional checkbox hides the OS cursor over it.
- **NDI is independent of this window.** You can send NDI without opening Program Out, and vice versa, because NDI is a network output, not a local display.

NDI output

Two independent senders, each with its own toggle:

- **Program Out:** the current slide.
- **Next Slide:** the upcoming slide, so a stage monitor or director's preview can show what's coming without the server compositing anything.

Both are proper NDI senders built against the official Vizrt SDK. Each **repeats its last frame once a second** as a keep-alive, so a static slide doesn't go stale for receivers expecting a steady feed. That means a frozen picture at the far end doesn't prove the sender's died.

Live capture (Keynote/PowerPoint)

Instead of sending a static pre-exported frame, you can capture the screen live, so animations, transitions and embedded video actually appear in the NDI output. Opt in per stream with the gear icon next to each NDI toggle. You can crop to part of the screen by percentage, which is handy for picking out just the "next slide" box from a Presenter Display that shows current, next and notes together.

It captures a whole physical display, not a window. Fullscreen presentation windows can't be enumerated through macOS's window-level capture sources, because they sit above the level that query reads. So pick the display the deck is on and crop if you need to.

This needs **Screen Recording permission** on macOS. Grant it in System Settings → Privacy & Security → Screen Recording, and restart the app.

On Windows it's offered for PowerPoint as well. I've only run it there in a Parallels VM, where Chromium's newer capture call failed and the app fell back to the older one.

Driving Keynote and PowerPoint

The app talks to a **currently-open document** in Keynote or PowerPoint itself, via AppleScript/JXA on macOS and PowerShell COM on Windows. Open the deck in Keynote or PowerPoint first, then connect.

macOS will ask for Automation permission the first time (System Settings → Privacy & Security → Automation). If you deny it, the bridge fails in ways that make the deck look empty. Grant it, and

grant Screen Recording too if you want live capture.

Things worth knowing:

- **Keynote page changes are polled at ~400 ms.** Advancing Keynote directly with a clicker is reflected back, but not instantly.
- **On macOS, PowerPoint frames are captured one slide at a time via the clipboard.** PowerPoint for Mac declares a bulk `save … as PNG/PDF` in its AppleScript dictionary, and in the version I tested it silently does nothing, so the app uses `copy object` and reads the image off the system clipboard instead. That's slower on a long deck, and **it uses the clipboard**, so don't be surprised by the clipboard changing under you while a deck loads.
- On Windows, `Slide.Export` does work, so it's a plain bulk loop.
- **Sections are read once, when the deck opens.** Editing sections mid-show won't be picked up.
- **On macOS PowerPoint, the media and section OSC commands are accepted and do nothing.** They aren't rejected, so a controller can't tell the difference between "no media on this slide" and "not supported on this platform".

Presenter notes

Notes are per-slide and auto-save to a **`.notes.json` file next to the PDF**.

It reads two file shapes: the bare `{"1": "note"}` map this app writes itself, and the richer sidecar written by presentation-converter, which carries provenance: the source deck, the engines used, and whether hidden slides shifted the page numbering.

Editing notes in this app keeps that provenance. If the file came from presentation-converter, the app merges your edits into it rather than overwriting it with a bare map, so the source-deck record doesn't get wiped the first time you fix a typo.

Why the hidden-slide detail matters: exporters drop hidden slides, so **slide count and page count aren't the same number**. If notes look off by a slide or two on a deck with hidden slides, that mapping is where to look.

Google Slides and Canva

Both need the unpacked Chrome extension in `extension/`. Load it at `chrome://extensions` → Developer mode → Load unpacked. Both relay through a local WebSocket bridge on port **9801**.

- **Google Slides** needs a **one-time OAuth client registration** in Google Cloud Console to fetch speaker notes. The ⚙ next to "Connect Google Slides…" walks you through it in the app and takes the client ID directly, with no file editing. You then have to **reload the extension** at

`chrome://extensions`, because Chrome only picks up manifest changes on reload. The full walkthrough is at `extension/OAUTH_SETUP.md`.

- **Canva needs no OAuth.** Its notes and slide position are scraped from the Presenter Window's DOM, because Canva has no public API for either. Open Present → Presenter view.

What that difference means in practice:

- **Google Slides notes lag the frame slightly.** The frame and index arrive straight away, and the notes come a beat later from the API.
- **Canva notes are scraped**, so they arrive with the frame, but scraping is fragile by nature. A Canva UI change can break notes without breaking anything else.
- **Canva only shows notes when there are some.** The element the app reads only exists once notes have content. An empty-notes slide has a placeholder prompt instead, which can't be read.

Remote control

From the Master Server

Connect to the Master Server's hub at `ws://<host>:9800`. The app registers by name, streams slide position and notes, and takes next/previous-slide commands from the server's Control Surface.

> ⚠ **There's no automatic reconnect.** If the link drops (a network blip, the server restarting, a switch rebooting), it stays disconnected until you reconnect it by hand. Watch the connection indicator during a show. The app will carry on presenting perfectly well while silently no longer being remote-controllable.

> ⚠ **Give every machine a distinct name.** The server identifies clients by name, so two laptops registering with the same name collapse into one entry on the server.

From OSC / Stream Deck

A UDP OSC address space at `/presentcommander/...`, with its own Companion module. Default ports: **listens on 35551**, sends feedback to **127.0.0.1:35550**. Off by default.

The full address list is in API.md's OSC control section. What catches people out:

- **Actions must be enabled.** While they're disabled, every message except `actions/enable` is **dropped silently**. This is the number one cause of "the button does nothing".
- **Feedbacks must be enabled** for anything to come back, except `feedbacks/refresh`, which always sends.

- `pause` / `resume` **do nothing unless timed auto-advance is already turned on.** They suspend an existing auto-advance timer. They don't start the feature.
- `black` / `white` / `laserpointer` **toggle when sent with no argument.** If a controller sends a malformed argument, it's treated as absent, so a badly-formed "blackout on" can turn blackout *off*.
- `goto/section` **is case-sensitive** and does nothing on an unknown name.
- **Nothing ever reports a refusal.** An unsupported or ignored address is silently dropped.

> ⚠ **OSC has no authentication.** Anything that can send a UDP datagram to port 35551 can jump slides, blank the screen, open and close Program Out, list and open files from the watched folder, and change the desktop wallpaper. Keep it on a control network.

The watched folder

Off by default. When it's on, OSC can open a deck by filename with no dialog, which is handy for a button wall that loads a particular deck on cue.

The folder is always set **relative to your home directory**. Only `.pdf`, `.key`, `.pptx` and `.ppt` are listed or openable, and any filename containing `/`, `\` or `..` is refused. A refusal is a silent no-op, not an error.

Other remote features

- **Laser pointer:** mirrors your mouse position over the "Now" preview onto Program Out as a glowing dot. It works whatever the source, since it's a display overlay.
- **Set wallpaper:** renders what's on screen and sets it as the desktop wallpaper on every connected monitor. macOS and Windows fully; **Linux is GNOME-only**.
- **Timed auto-advance:** "advance every N seconds". It **stops at the last slide rather than looping**.

Troubleshooting

Symptom	Cause
OSC does nothing at all	Actions are disabled. Send `/presentcommander/actions/enable` first (Remote control).
No OSC feedback	Feedbacks are disabled, or the feedback host/port is wrong. `feedbacks/refresh` always sends — use it to test.
`pause` **does nothing**	Auto-advance was never turned on (Remote control).
Blackout turned off when I asked for on	A malformed argument is treated as absent, and no argument means toggle (Remote control).
`goto/section` **does nothing**	Case mismatch, or the source has no sections at all — only PDF and PowerPoint-on-Windows do (Choose your source first — the capabilities differ).
Media commands do nothing	Only PowerPoint on Windows, and only with a live slideshow running separately. On macOS they're accepted and ignored (Driving Keynote and PowerPoint).
Notes garbled / non-ASCII wrong	Read `notes-utf8` (blob) rather than `notes` (string) (API.md: OSC control).
Server link silently stopped working	It dropped and there is no auto-reconnect (Remote control).
Two laptops show as one on the server	They registered with the same name (Remote control).
Keynote/PowerPoint bridge acts like the deck is empty	macOS Automation permission was denied (Driving Keynote and PowerPoint).
Live capture is black or stalls	Screen Recording permission, or the wrong display picked. Capture is per-display, not per-window (NDI output).
PowerPoint deck loads slowly on macOS, clipboard keeps changing	Expected — frames come one at a time through the clipboard because bulk export is a silent no-op there (Driving Keynote and PowerPoint).
Sections changed mid-show and didn't update	They're read once at open (Driving Keynote and PowerPoint).
Google Slides notes are blank or late	They arrive a beat after the frame, and need the OAuth client ID plus an extension reload (Google Slides and Canva).
Canva notes blank	Canva only renders the element the app reads when notes are non-empty (Google Slides and Canva).

Symptom	Cause
Extension stops relaying after a while	MV3 service worker suspension. The extension reconnects defensively on every message for this reason; if it persists, reload it at `chrome://extensions`.
Notes off by a slide or two	Hidden slides shift page numbering — slide count ≠ page count (Presenter notes).
NDI receiver shows a frozen frame	Not necessarily dead — the sender repeats the last frame every second by design (NDI output).
macOS says the app is damaged	Not the released build — those are signed and notarised. A self-built or pre-notarisation copy is quarantined. Use the current release; UNSIGNED.md shows how to check a copy is the notarised one.

Before a show

1. Pick the source type carefully (Choose your source first — the capabilities differ). PDF if you can.
2. Grant macOS **Automation** and **Screen Recording** permissions and restart, if you need Keynote/PowerPoint or live capture (Driving Keynote and PowerPoint).
3. Open the deck in its app *first*, then connect the bridge.
4. Check the notes came across, especially on a deck with hidden slides (Presenter notes).
5. Give the machine a distinct name before connecting to the server (Remote control).
6. Enable OSC **actions and feedbacks** and prove one button works before you rely on the wall (Remote control).
7. Check the NDI receivers actually see both senders.
8. Keep the control network private, as neither OSC nor the server link is authenticated (Remote control).

See also

- API.md: the full OSC address space, server protocol, extension bridge, sidecar format
- DEVELOPING.md: building it
- README: feature detail, architecture, prior art